

SCIENTIFIC RESEARCH PAPER

EraSeMap: verifiable personal-data erasure from distributed systems and machine-learning models

A unified FIND–ERASE–PROVE algorithm

Author	Nurlanuly Dulat, Grade 9B
Affiliation	Specialized Bilim-Innovation Lyceum-Boarding School of the Almaty City Education Department
Direction	Direction II — mathematical modelling of economic and social processes
Section	Section: Computer Science
Supervisor	Smagul Yerzat Aidynuly, Computer Science Teacher

Almaty, 2026

Contents

Abstract.....	3
1. Introduction.....	4
2. Related work and novelty boundary.....	4
3. System model.....	5
4. The unified EraSeMap algorithm.....	6
5. Formal properties.....	7
6. Implementation.....	7
7. Experimental methodology.....	8
8. Results.....	8
9. Discussion.....	9
10. Limitations and threats to validity.....	10
11. Ethics and responsible use.....	10
12. Reproducibility.....	10
13. Conclusion.....	11
References.....	11
Appendices.....	12

Abstract

Deleting a primary record does not establish that personal data can no longer be used. Copies may remain in caches, indexes, replicas, exports, or backups; derivatives may remain as biometric templates and vectors; learned influence may remain in a trained model. A deleted object may also reappear after restore, synchronization, rebuild, or model redeployment.

I developed and tested EraSeMap as one algorithm with three stages. **FIND** locates known and bounded, non-obvious paths. **ERASE** selects a minimum sufficient set of physical actions and removes learned influence from the model (machine unlearning). **PROVE** repeats future operations and permits a certificate only after every required check passes. The algorithm returns COMPLETE_WITHIN_ENVELOPE, INCOMPLETE, or UNVERIFIED; missing evidence is never treated as success.

In a pre-registered 60-case transfer, EraSeMap produced 0 false COMPLETE decisions versus 5 for a full node audit and 45 for the service's ordinary status. Active recovery-graph search used 7 probes versus 13 for random testing and 49 for exhaustive testing; greedy also used 7. In 20 paired real-process trials, the selected plan was 17.64 times faster on average and wrote 94.62% fewer bytes than rebuilding everything. Temporal verification detected 30/30 latent risks and produced 0/30 post-control recurrences. Exact solvers matched exhaustive oracles in 3,072/3,072 and 16,384/16,384 configurations. Fast influence-removal candidates on Qwen2.5-1.5B failed all required thresholds together, so full retraining from scratch remains the safe fallback. Results are limited to registered topology and listed transitions; production FaceID/eGov deployment and an independent hidden result are not claimed.

Keywords: verifiable erasure, machine unlearning, data lineage, temporal replay, biometrics, fail-closed, deletion certificate.

1. Introduction

Erasure rights and organizational policies require removing one person's data. In practice, one event creates a chain of artifacts: a source record, normalized profile, biometric template, search vector, cache, log, export, backup, and model parameters. Each component may issue a correct local receipt while the overall system continues to use the person.

Existing research addresses parts of the problem. Data lineage represents provenance [9]. Machine unlearning reduces training-example influence [1, 2], while verification-of-unlearning studies the strength of resulting evidence [3–8]. Deletion-policy systems and patents address lineage, backup, and receipts [10–12]. A local proof for one component, however, does not automatically cover physical derivatives, a model, and future regeneration together.

The research question is:

How can one issue a verifiable subject-level erasure verdict that jointly covers physical copies, derivatives, learned influence, and registered future recovery paths?

This work contributes:

1. one three-stage fail-closed FIND–ERASE–PROVE algorithm;
2. a typed model of physical, derivative, and model-channel evidence;
3. exact minimum-cost action selection with a counterexample when erasure is infeasible;
4. temporal replay that blocks certification when regeneration is possible;
5. mandatory comparison of machine unlearning with exact retraining and preservation of negative results;
6. reproducible layered evaluation on bounded graphs, real local processes, stock services, face data, and an open Qwen2.5-1.5B model.

2. Related work and novelty boundary

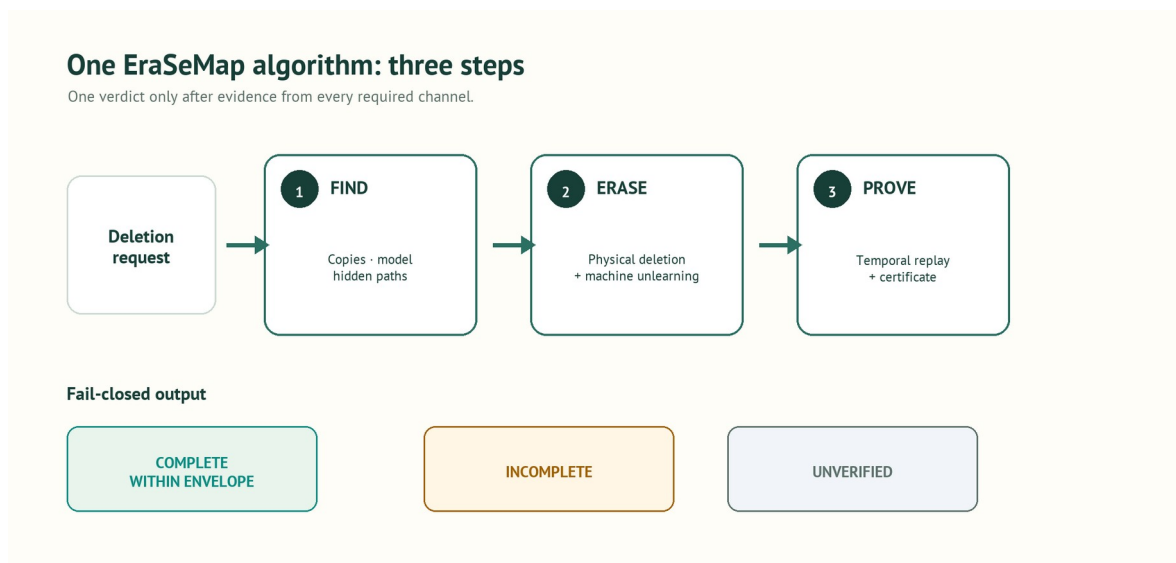


Figure 1. One EraSeMap algorithm: FIND, ERASE, and PROVE. Author-generated from the EraSeMap protocol.

EraSeMap does not claim to invent lineage graphs, set cover, active testing, temporal model checking, machine unlearning, or digital signatures. Its narrower contribution is their composition into one subject-level decision rule in which no local receipt can independently create a positive verdict.

Unlike a conventional lineage audit, the relevant object is a usable path rather than only a typed node. Unlike model-only unlearning, reducing learned influence does not close a backup, cache, or vector index. Unlike snapshot testing, present absence does not establish that restore cannot bring the object back. Unlike an unbounded promise, COMPLETE_WITHIN_ENVELOPE explicitly scopes its guarantee to the registered system.

3. System model

3.1 Typed graph

Let the system be represented by

$$G = (V, E, \tau, s),$$

where V contains artifacts and services, E contains provenance or recovery operations, τ assigns node and edge types, and s binds evidence to a subject. For subject u , $R_u(G)$ is the set of usable residual paths. A path is active if its terminal artifact can store, return, recognize, or regenerate information about u .

Mandatory channels C_u include:

- physical: source and backup artifacts;
- derivative: templates, embeddings, indexes, and exports;
- model: learned influence;
- privacy: preregistered privacy-proxy checks;
- utility: retained-subject or retained-task quality;
- temporal: no return after future operations;
- coverage: sufficient registered instrumentation.

Each local verifier returns PASS, FAIL, or UNKNOWN.

3.2 Three verdicts

COMPLETE_WITHIN_ENVELOPE:

$R_u(G)$ is empty, every mandatory channel passes, discovery evidence is valid, and temporal replay is safe.

INCOMPLETE:

a concrete active or regenerating path exists.

UNVERIFIED:

available evidence proves neither a residual nor complete erasure.

Three-valued logic prevents absent evidence from being converted into false success.

4. The unified EraSeMap algorithm

4.1 FIND

FIND first replay-audits the registered graph. When the map may be incomplete, an adapter performs safe synthetic probes: it enables selected recovery operations and observes where and when a test subject reappears. After every probe, only graphs consistent with the trace survive. The stage returns one graph, a complete observable path class, OUT_OF_HYPOTHESIS, or UNVERIFIED.

Let H be a finite version space and $\text{Obs}(q, h)$ be the trace of query q under hypothesis h . After observing o :

$$H' = \{h \text{ in } H : \text{Obs}(q, h) = o\}.$$

Exact one-step minimax selects the permitted probe that minimizes the largest next class with a deterministic tie-break. This is a local guarantee, not optimality of the full adaptive tree.

4.2 ERASE

Let A be a finite action catalog, $c(a) \geq 0$ its declared costs, and $\text{Apply}(G, B)$ the post-action system for subset B . A plan is feasible when every action is permitted and replay closes every active path and mandatory channel. EraSeMap selects

$$B^* = \arg \min_{a \in B} \sum c(a)$$

subject to Feasible(B) = true.

A stable lexicographic rule breaks equal-cost ties. If no complete permitted plan exists, the algorithm returns INCOMPLETE or UNVERIFIED, never a partial receipt.

The model branch is inside ERASE. A candidate unlearning method is compared with exact retraining on forgetting, retained utility, privacy proxy, deletion-matched distance, and recurrence after reload. The model pass is the conjunction of frozen gates. Failure of one gate invokes exact retraining or leaves the model channel incomplete.

4.3 PROVE

Let q_0 be the post-ERASE state and δ the registered future transitions. Safety requires:

$$\text{Safe}(u) \text{ iff for every } q \text{ in } \text{Reach}(q_0, \delta): \text{Residual}(u, q) = \text{false}.$$

If replay finds a recurrence witness, PROVE returns INCOMPLETE with a shortest counterexample. If transition coverage is unknown, it returns UNVERIFIED. Only safe replay permits certificate-ready status.

4.4 Unified rule

Let P denote closure of physical/model paths, D valid discovery evidence, and T temporal safety:

$$\text{COMPLETE_WITHIN_ENVELOPE} \text{ iff } P \text{ and } D \text{ and } T.$$

5. Formal properties

The Lean project checks four conditional properties without sorryAx:

1. replayed completion excludes represented residual paths under topology completeness and sound local verifiers;
2. the exact finite selector chooses a feasible plan no more costly than another listed feasible plan;
3. observed transition coverage lifts snapshot absence to all reachable registered states;
4. the exact temporal selector is safe and minimum-cost under an explicit feasibility-soundness obligation.

Checked counterexamples show that a hidden residual is possible without topology coverage and that a passed channel is meaningless without verifier soundness. The Python implementation is separately compared with exhaustive oracles; Lean does not prove Python semantics, adapters, or completeness of an actual organization.

6. Implementation

The public Python entry point `run_erasemap` returns three stage results and one verdict. Service adapters are separated from the pure verifier, preventing unreviewed network or destructive calls. Evidence bundles are bound by hashes and, where required, Ed25519 signatures. The CLI builds an offline-verifiable showcase and reports.

The internal names PCUG, GhostGraph, CDC, RSE, and MSC are retained only for evidence tracing and formal namespaces. They do not represent five competing user-facing algorithms.

7. Experimental methodology

7.1 Primary metric

The critical risk is false-complete rate:

FCR = false COMPLETE / actually incomplete cases.

Secondary metrics include probes, action cost, wall time, bytes written, retained loss, recurrence, oracle mismatches, forgetting, utility, and privacy advantage.

7.2 Evidence layers

1. Mechanism stress distinguishes typed-node from channel-aware path semantics.
2. Stock-service transfer uses digest-pinned Keycloak, MLflow, and Qdrant in 60 frozen cases.
3. Bounded hidden graphs compare active minimax, greedy, random, and exhaustive FIND.
4. The measured multi-service experiment compares targeted ERASE with rebuild-all on real PostgreSQL, Redis, Qdrant, encrypted backup, and a ridge model.
5. The temporal lab tests delayed restore, synchronization, cache/index rebuild, and coverage gaps.
6. Face experiments evaluate bounded unlearning and retained-user privacy.
7. Qwen-TOFU evaluates adapter-level learned influence on a real open 1.5B model.

Protocols, seeds, gates, and source hashes were frozen before the corresponding confirmation runs. Failed results are retained rather than rewritten.

8. Results

Test	EraSeMap	Baseline	Interpretation
Stock-service false COMPLETE	0/60	typed 5/60; native 45/60	FIND/coverage blocks local-receipt errors
Hidden-graph probe budget	7	greedy 7; random 13; exhaustive 49	win over random/exhaustive, tie with greedy
Exact action conformance	3,072/3,072	exhaustive oracle	0 mismatches in bounded domain
Targeted execution time	5.67%	rebuild-all 100%	17.64x geometric speedup
Written bytes	5.38%	rebuild-all 100%	94.62% reduction
Temporal risk detection	30/30	snapshot 0/30	PROVE tests a stronger future claim

Test	EraSeMap	Baseline	Interpretation
Post-control recurrence	0/30	no control 30/30	registered controls close frozen risks
Exact temporal conformance	16,384/16,384	exhaustive oracle	0 mismatches in bounded domain

All 20 paired real-process cases preserved completion and retained data. The frozen transfer also recorded zero retained loss and zero post-control recurrence.

8.1 Model channel

Bounded face experiments provided positive project-authored evidence, including preregistered sequential retained-user privacy gates. This is not certified privacy.

Qwen v1 evaluated three seeds and returned FAIL: the candidate approached exact retraining on some metrics but did not simultaneously meet forgetting and world-utility requirements. Qwen v2 used author-disjoint development selection and five untouched confirmation seeds. It was at least 30.48x faster and had zero recurrence after reload, but overscrubbing and several exact-matching/utility gates failed. The fast-candidate model verdict therefore remains incomplete; exact adapter retraining is the reference fallback. This demonstrates the fail-closed policy rather than weakening it.

9. Discussion

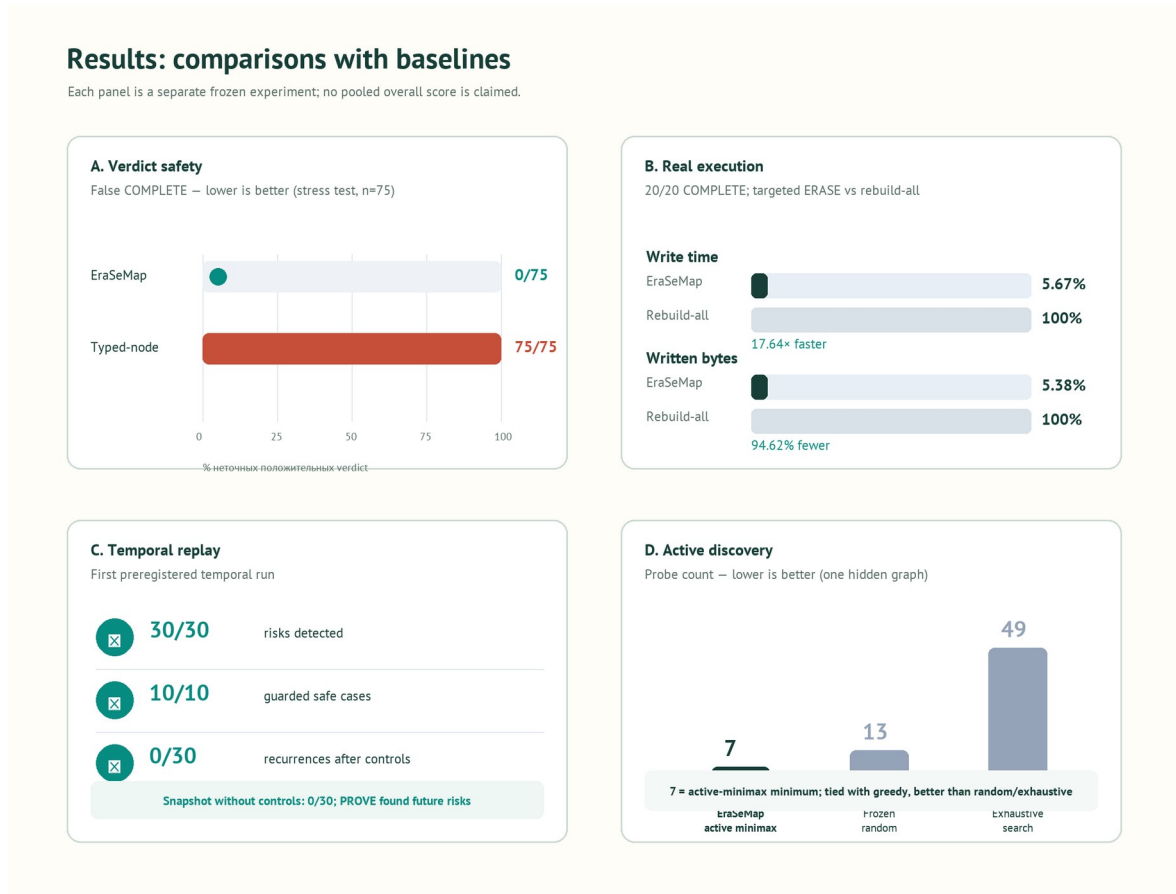


Figure 2. Four measured properties of the unified EraSeMap algorithm: safety, efficiency, temporal verification, and active discovery. Author-generated from the frozen results.

The main result is not universal superiority of one numerical method. EraSeMap connects different evidence types so that a weak local metric cannot close the entire erasure request. A low membership score does not delete a backup, and an empty snapshot does not rule out future restore.

Active FIND reduces diagnostic probes, although the strong greedy baseline tied on the frozen catalog. Targeted ERASE shows substantial systems savings over rebuild-all, but on one local machine. PROVE extends the claim over time, but only under transition coverage. Every strong result therefore has an explicit boundary.

10. Limitations and threats to validity

- Hidden graphs, mappings, faults, and execution are project-authored; no accepted external run exists.
- Stock services are real, but subjects are synthetic or public rather than customer records.
- A bounded catalog does not cover arbitrary secret infrastructure.
- A local verifier can be wrong; a signature protects integrity, not measurement truth.
- Performance was measured on one local configuration.
- The face privacy experiment is not certified privacy.

- Qwen experiments address adapter influence, not deletion from Qwen pretraining.
- No production FaceID, eGov, KYC, bank, or government deployment was conducted.

11. Ethics and responsible use

Public demonstrations use synthetic identities or open datasets. Active probes must be authorized, isolated, and harmless to retained users. EraSeMap must not be used for false compliance: certificate scope, unknown channels, and instrumentation gaps must remain visible.

12. Reproducibility

```
python -m venv .venv
.venv/bin/pip install -e '[dev,real]'
./scripts/reproduce_release.sh core
.venv/bin/erasemap showcase --repo-root . --output outputs/jury-showcase-v1
```

The release gate includes Ruff, strict mypy, pytest with at least 90% coverage, package build, frozen evidence verifiers, oracle comparisons, and Lean. The external hidden challenge has a separate blind handoff; its current result is NOT_COLLECTED.

13. Conclusion

EraSeMap turns data erasure from a local command into one verifiable process. FIND locates registered and bounded-hidden paths. ERASE closes physical and model branches with a minimum sufficient plan. PROVE tests future regeneration and permits a scoped certificate. Formal properties are conditional and explicitly bounded; real negative ML results are preserved. The design supports an honest path toward FaceID/eGov/KYC-like systems, while production and independent validation remain future work rather than present claims.

References

- [1] Y. Cao and J. Yang. Towards Making Systems Forget with Machine Unlearning. IEEE S&P, 2015. DOI: 10.1109/SP.2015.35.
- [2] L. Bourtole et al. Machine Unlearning. IEEE S&P, 2021. arXiv:1912.03817.
- [3] D. M. Sommer et al. Towards Probabilistic Verification of Machine Unlearning. arXiv:2003.04247, 2020.
- [4] J. Weng et al. Proof of Unlearning: Definitions and Instantiation. arXiv:2210.11334, 2022.
- [5] T. Eisenhofer et al. Verifiable and Provably Secure Machine Unlearning. SaTML, 2025.
- [6] R. Chourasia and N. Shah. Forget Unlearning: Towards True Data-Deletion in Machine Learning. ICML, 2023.
- [7] B. Zhang et al. Verification of Machine Unlearning is Fragile. ICML, 2024.
- [8] A. Koloskova et al. Certified Unlearning for Neural Networks. ICML, 2025.

- [9] T. Lebo, S. Sahoo, and D. McGuinness, eds. PROV-O: The PROV Ontology. W3C, 2013.
- [10] US20 220 414 070A1. Tracking Data Lineage and Applying Data Removal to Enforce Data Removal Policies, 2022.
- [11] US11 120 156B2. Privacy Preserving Data Deletion, 2021.
- [12] US12 456 052B2. Systems and Methods for Facilitating Verifiability of ML Model Unlearning, 2025.
- [13] NIST SP 800-63A-4. Digital Identity Guidelines: Identity Proofing and Enrollment, 2025.
- [14] EraSeMap. Public repository and evidence archive, v0.5.0, 2026: <https://github.com/nazkari86-lab/erasemap>.
- [15] V. Chakraborty et al. Meaningful Data Erasure in the Presence of Dependencies. PVLDB 18(10), 2025.

Appendix A. Notation

Symbol	Meaning
$G=(V,E,\tau,s)$	registered typed erasure graph
u	subject of one erasure request
$R_u(G)$	active subject residual paths
C_u	mandatory verifier channels
A	finite candidate-action catalog
B^*	minimum feasible action set
H	bounded recovery-graph version space
δ	registered future transitions
FCR	false-complete rate

Appendix B. Claim–evidence map

Claim	Evidence	Boundary
Replayed completion is conditionally sound	Lean theorem and counterexamples	topology/verifier assumptions required
Exact ERASE is minimum-cost	Lean plus 3,072/3,072 oracle matches	finite registered catalog

Claim	Evidence	Boundary
PROVE is temporally safe	Lean plus 16,384/16,384 oracle matches	registered transition coverage
FIND reduces probe budget	7 vs 13 random vs 49 exhaustive	finite project-authored catalog; greedy tie
Targeted ERASE is cheaper than rebuild-all	17.64x and 94.62% fewer bytes	one local system, synthetic identities
Fast Qwen unlearning succeeds	not established: v1/v2 FAIL	exact retraining remains fallback
External hidden generalization	protocol ready	NOT_COLLECTED
Production FaceID/eGov	pilot protocol only	not established